

University of Asia Pacific
Department of Computer Science and Engineering
REPORT: 02

Course Code: CSE 402

Course Title: Operating System Lab

Submitted by:

Name: Rajash Majumdar

Student ID:23101144

Semester : 4th Year 1st Semester

Section: C2

Submitted to:

Atia Rahman Orthi,
Lecturer
University of Asia Pacific

Problem Statement:

LAB 2 Worksheet:

Implement FCFS CPU Scheduling for the following scenario where No of processes = 5 and Process informations are-

P id	AT	BT
P0	3	1
P1	5	3
P2	2	2
P3	1	2
P4	6	3

Calculate CT, TAT, WT, AVG TAT and AVG WT. Also represent the process execution sequence.

Problem Solving Approach using Python:

```
processes = [  
    ["P0", 3, 1],  
    ["P1", 5, 3],  
    ["P2", 2, 2],  
    ["P3", 1, 2],  
    ["P4", 6, 3]  
]  
processes.sort(key=lambda x: x[1])  
  
current_time = 0  
results = []  
execution_order = []
```

```

for p in processes:
    p_id, at, bt = p

    if current_time < at:
        current_time = at

    ct = current_time + bt
    tat = ct - at
    wt = tat - bt

    current_time = ct
    execution_order.append(p_id)
    results.append([p_id, at, bt, ct, tat, wt])

print("Execution Sequence:", " ->
".join(execution_order))
print("Process\tAT\tBT\tCT\tTAT\tWT")

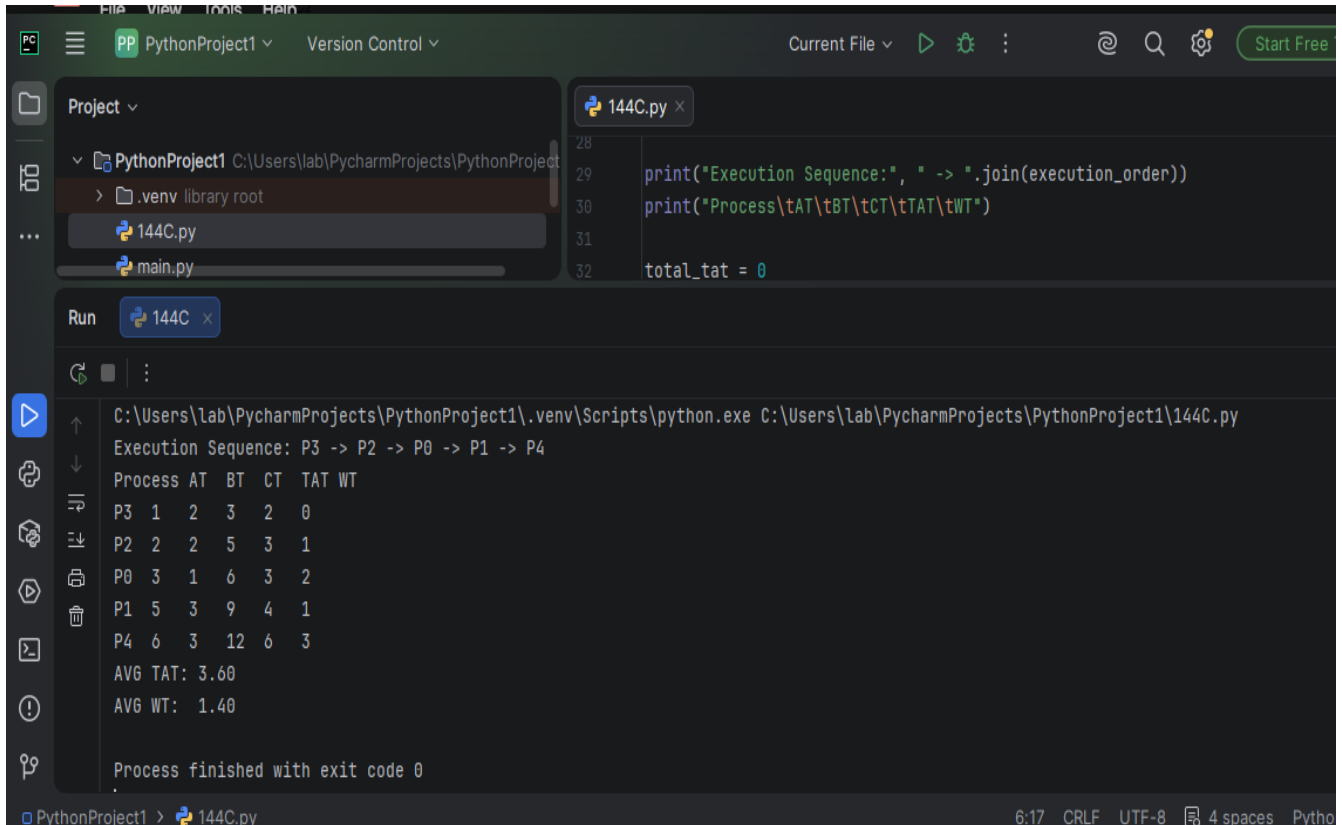
total_tat = 0
total_wt = 0

for row in results:
    p_id, at, bt, ct, tat, wt = row
    total_tat += tat
    total_wt += wt
    print(f"{p_id}\t{at}\t{bt}\t{ct}\t{tat}\t{wt}")

n = len(processes)
print(f"AVG TAT: {total_tat / n:.2f}")
print(f"AVG WT: {total_wt / n:.2f}")

```

Output of the code:



The screenshot shows the PyCharm IDE interface. The top toolbar includes a 'Run' button (a green play icon). The left sidebar shows the project structure with '144C.py' selected. The main editor window displays the code for '144C.py', which includes a print statement for the execution sequence and a table of process metrics. The bottom panel shows the output of the script, which includes the execution sequence and the same table of process metrics.

```
28
29 print("Execution Sequence:", " -> ".join(execution_order))
30 print("Process\tAT\tBT\tCT\tTAT\tWT")
31
32 total_tat = 0
```

Run 144C x

C:\Users\lab\PycharmProjects\PythonProject1\.venv\Scripts\python.exe C:\Users\lab\PycharmProjects\PythonProject1\144C.py

Execution Sequence: P3 -> P2 -> P0 -> P1 -> P4

Process	AT	BT	CT	TAT	WT
P3	1	2	3	2	0
P2	2	2	5	3	1
P0	3	1	6	3	2
P1	5	3	9	4	1
P4	6	3	12	6	3

AVG TAT: 3.60
AVG WT: 1.40

Process finished with exit code 0

Git repository link:

<https://github.com/Rajash144/FCFS-CPU-Scheduling-Lab2>

Discussion:

While implementing the FCFS scheduling algorithm, students may face a few common problems. They might forget to sort the processes by arrival time, resulting in an incorrect execution order. Mistakes in calculating CT, TAT, or WT are also common. Another challenge is handling CPU idle time correctly when no process has arrived. Careful implementation and checking each calculation can help avoid these errors.

Conclusion:

The FCFS CPU Scheduling algorithm was successfully implemented using Python. The program correctly determined the execution sequence and calculated CT, TAT, WT, AVG TAT, and AVG WT. This experiment helped in understanding the basic concept and working process of the FCFS scheduling algorithm.